

1. Application Title and Problem Addressed

Petal & Stem - Florist Appointment System The Part 2 analysis addressed critical security vulnerabilities, weak data validation, and a monolithic codebase by migrating the system from a client-side localStorage application to a secure, modular Node.js client-server architecture.

2. Group Members

- H.G. Dineth Vishwa (Index: ICT24830)
- Y.M. Paronavithana (Index: ICT24899)
- R.I.B. Sadeep Mihiran (Index: ICT24885)
- W.M. Ravindu (Index: ICT24915)
- K.K.P. Akalanka (Index: ICT24806)

3. Group Leader

K.K.P. Akalanka (Index: ICT24806)

4. Reference to Original Part 1 Baseline

Original branch: main / baseline-part-1 (Available in the group's submission repository).

5. Summary of Original Features

- **Completed:** Single Page Application (SPA) tab navigation, dynamic booking form with cascading dropdowns, visual monthly calendar with indicator dots, real-time dashboard for status updates, and automated toast notifications.
- **Incomplete:** Authentic administrative protection, server-side data persistence, and robust date validation preventing past bookings.

6. Practical and Learning Problems Experienced

- **Lack of understanding of generated code:** The AI generated a single monolithic index.html file containing HTML, CSS, and JavaScript, making it difficult to maintain or isolate specific logic bugs.
- **Difficulty distinguishing real functionality from mock behavior:** The initial application appeared fully functional, but the data persistence relied entirely on the browser's localStorage, meaning clearing the browser cache wiped the entire database.
- **Uncertainty about security:** The team lacked experience in enforcing authorization, resulting in a dashboard that anyone could access.

7. Testing Approach Used in Part 2

- **Boundary Testing:** Attempted to input historical dates and incomplete custom fields into the booking form to evaluate validation strengths.
- **Authorization Testing:** Attempted to access the dashboard and analytics interfaces via direct DOM manipulation and URL routing without logging in.

- **Regression Testing:** Ensured the calendar UI and dynamic dropdowns continued to function correctly after separating the code into multiple files and attaching the Node.js backend.

8. Completed Defect Table

ID	Area	Steps/test performed	Expected result	Actual result	Evidence	Severity	Likely cause	Fix or proposed solution	Retest result
01	B. Validation	Select a date from the previous year in the booking form.	Form rejects input and displays an error.	Form accepts the past date and saves it.	Screenshot of past booking	High	Relied only on basic HTML5 validation.	Implement JS validation comparing input to new Date().	Pass. Form rejects past dates.
02	C. Security	Navigate directly to the "List & Status" dashboard tab.	Access denied; prompt for admin credentials.	Full access granted to view all customer PII and change statuses.	Screenshot of exposed dashboard	Critical	SPA lacked an authentication gateway.	Create a login page and a Node.js backend to verify credentials before serving.	Pass. App locked behind login screen.

ID	Area	Step/s/test performed	Expected result	Actual result	Evidence	Severity	Likely cause	Fix or proposed solution	Retest result
								ng data.	
03	G. Code Quality	Inspect source code structure.	Separation of concerns (HTML, CSS, JS).	All logic combined into one 500+ line index.html file.	Source code file	Low	All prioritized rapid deployment over maintainability.	Refactor into index.html, style.css, and app.js.	Pass. Files cleanly separated.

9. Code and Architecture Analysis

The Part 1 baseline violated the separation of concerns by mixing interface styling, business logic, and local data access within a single file. The application has been refactored into a client-server architecture using an Express.js backend (server.js) to handle API routes, while the frontend is modularized into index.html, style.css, and app.js.

10. Problems Corrected

- **Authentication Added:** Implemented a sleek, forced login gateway that intercepts all protected routes. *Evidence:* User is redirected to a login card requiring specific credentials before the navigation bar is rendered.
- **Date Validation Fixed:** Added strict JavaScript validation (dateInput.min = todayStr; and an active check on submit) to prevent historical bookings.
- **Data Persistence Upgraded:** Replaced client-side localStorage with a Node.js Express server utilizing the fetch() API for asynchronous data operations.

11. Unresolved Problems and Proposed Solutions

- **In-Memory Database:** The current Node.js server stores data in a temporary array, meaning data is lost if the server restarts. *Solution:* Requires time and access to integrate a persistent database like MongoDB.

- **Plaintext Passwords:** The authentication bypasses JWT tokens and password hashing.
Solution: Requires implementing bcrypt and jsonwebtoken libraries for production-level security.

12. Link/Files for Improved Version

Improved version available in the part-2-final branch of the group repository.

13. Part 2 AI Interaction and Generation Record

- **Prompt 1:** "Act as a professional web developer and implement dismiss above mistakes and improve security. Clearly separate files to index.html, style.css and app.js I need separate database it should be secure. Form should reject past dates. Access should be restricted to admin staff."
- **Prompt 2:** "Use node.js"
- **Prompt 3:** "Create a logging interface like this how I do this [attached image]"
- **Prompt 4:** "There is a problem logging interface appear each page like this vanish after enter username and password [attached image]"

14. Lessons Learned

The group learned that AI application generators prioritize visual completeness over architectural security. We discovered that a frontend-only web application can never truly protect sensitive data, and that fundamental web-development knowledge regarding server-client relationships, API routing, and CSS flexbox scoping is strictly necessary to evaluate, debug, and secure AI-generated code.